

Object Mapping with AutoMapper and Email Functionality with MailKit in Your .NET Application

Simplifying data transformation and email delivery in modern .NET apps



AutoMapper



MailKit



.NET

Agenda

1. Why object mapping matters
 2. DTOs, ViewModels, and Entities
 3. What AutoMapper is
 4. AutoMapper configuration
 5. Mapping profiles
 6. Why email functionality matters
 7. What MailKit is
 8. Sending emails with MailKit
 9. Best practices
-



Why Object Mapping Matters

Applications often move data between models:



Manual Mapping is...



⊗ Repetitive



⊗ Error-prone



⊗ Maintenance Overhead

Entities, DTOs, and ViewModels



Entity

Represents
database structure



DTO

Transfers data
between layers
or APIs



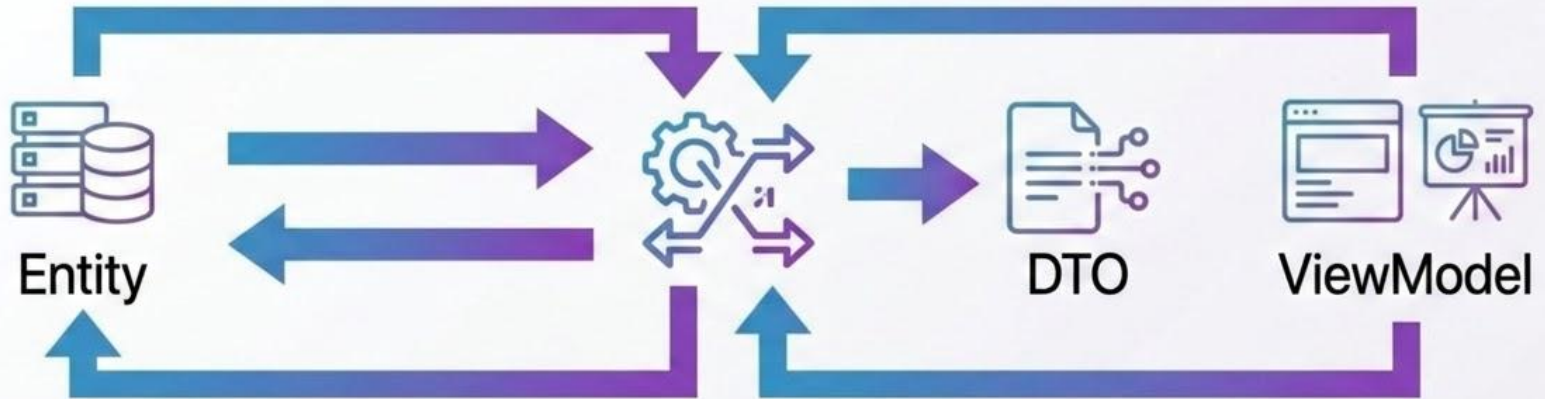
ViewModel

Represents data
needed by the UI

Key idea: Each model has a different responsibility. 

What Is AutoMapper?

AutoMapper is a .NET library that automatically maps from one object type to another.



Key idea: It reduces repetitive mapping code. 

Manual Mapping vs AutoMapper

Manual Mapping

```
var userDto = new UserDto {  
    Id = user.Id,  
    FullName = user.FullName,  
    Email = user.Email  
};
```

With AutoMapper

```
var userDto =  
    _mapper.Map<UserDto>(user);
```

Key idea: AutoMapper makes mapping cleaner and easier to reuse. 

AutoMapper Setup

Step 1: Installation 📦

Step 2: Configuration ⚙️

1. Install AutoMapper via NuGet. 📦
2. Register in the DI container. ⚙️

```
builder.Services.AddAutoMapper(typeof(Program));
```

⚙️ **Key idea:** Once registered, AutoMapper can be injected into services and controllers.

Mapping Profiles

Profiles define how objects are mapped.

```
public class UserProfile : Profile
{
    public UserProfile()
    {
        CreateMap<User, UserDto>();
        CreateMap<CreateUserViewModel, User>();
    }
}
```



Key idea: Profiles centralize mapping configuration.

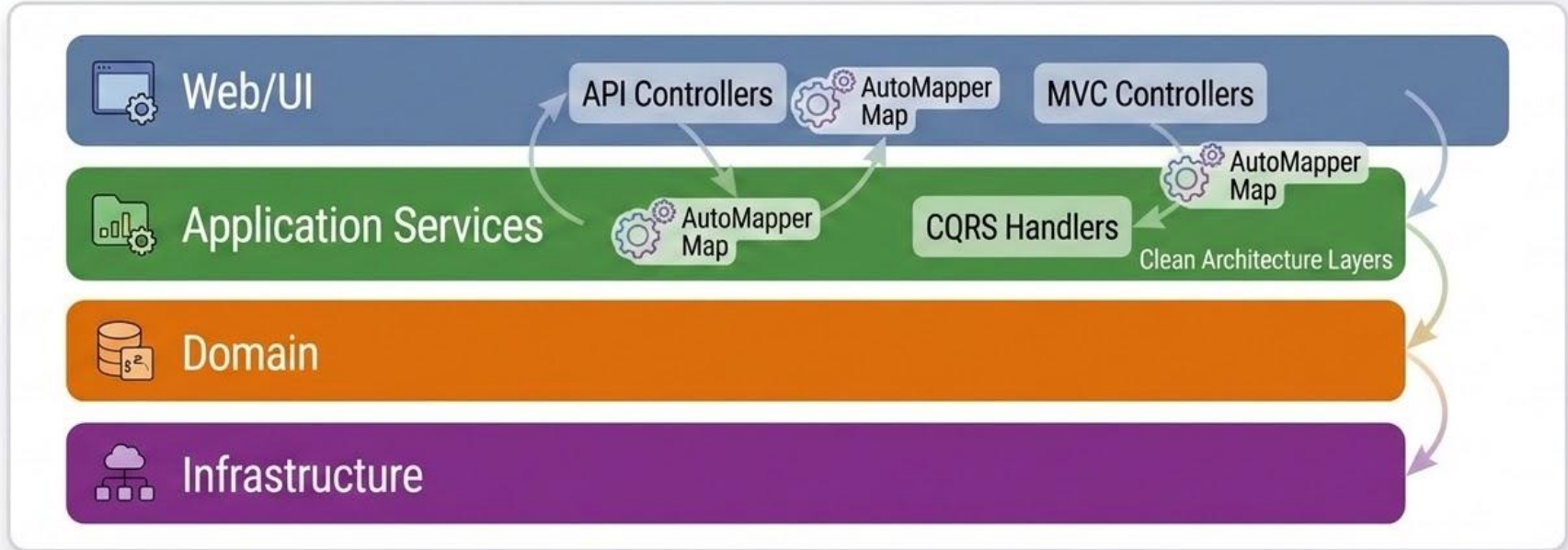
Custom Mappings

AutoMapper allows custom mapping rules when property names or formats are different.

```
CreateMap<User, UserDto>()  
    .ForMember(dest => dest.Name,  
        opt => opt.MapFrom(src => src.FullName));
```

 **Key idea:** Use custom mappings when automatic conventions are not enough.

Where to Use AutoMapper



 **Key idea:** Use AutoMapper to move data between layers without exposing internal models.

AutoMapper Best Practices



☐ Keep profiles organized by feature or module.



☐ Avoid complex business logic inside mappings.



☐ Validate mapping configuration.



☐ Use DTOs to avoid exposing entities.

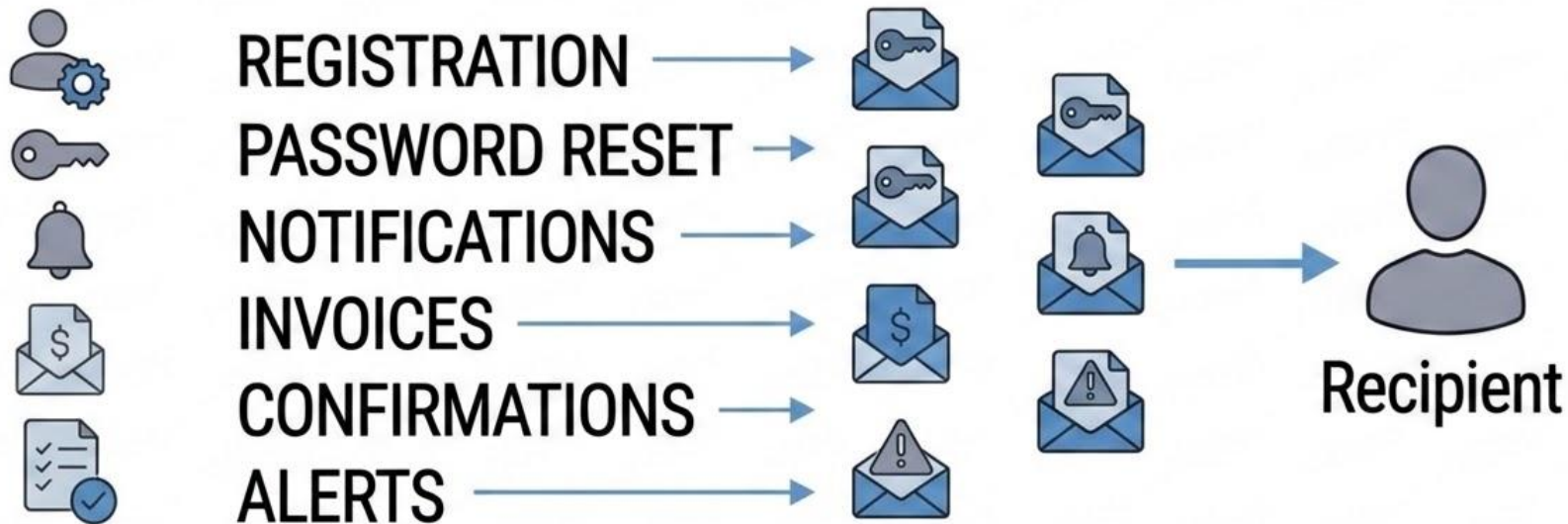


☐ Keep mappings simple and predictable.



Key idea: Follow best practices to ensure maintainable and reliable mappings.

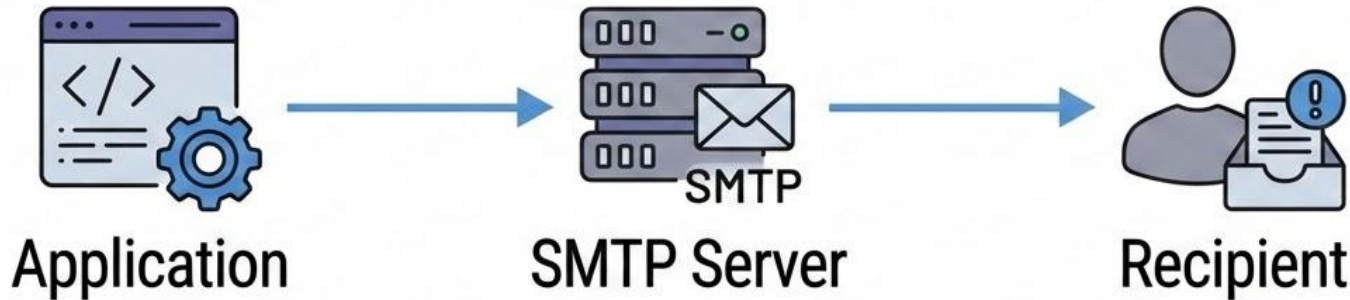
Why Email Functionality Matters



Key idea: Email is a common communication channel between the application and the user.

What Is MailKit?

MailKit is a modern .NET library used to send and receive email through protocols such as SMTP, IMAP, and POP3.



Key idea: For application email delivery, SMTP is commonly used.

Email Configuration



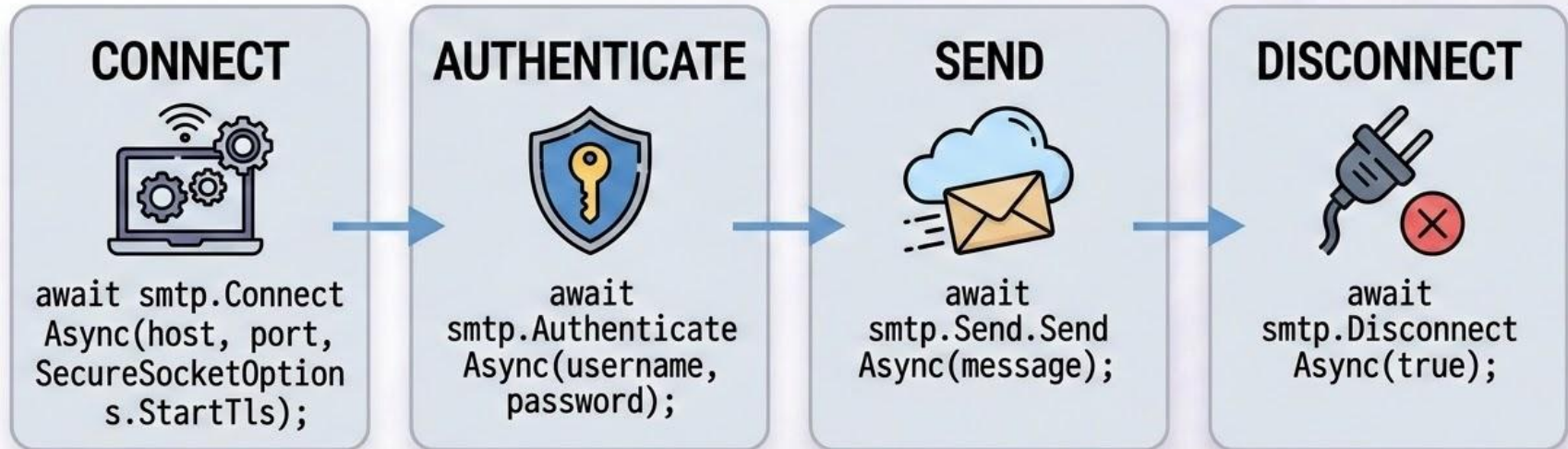
```
{  
  "MailSettings": {  
    "Host": "smtp.example.com",  
    "Port": 587,  
    "UserName": "noreply@example.com",  
    "Password": "your-password",  
    "FromEmail": "noreply@example.com"  
  }  
}
```



Key idea: Never hardcode credentials directly in the code.

Sending Emails with MailKit

MailKit sends emails using an SMTP client.



Key idea: Connect, authenticate, send, and disconnect.

Creating an Email Service

A clean application should hide email logic behind a service.



Email Service
Abstraction

```
public interface IEmailService
{
    Task SendEmailAsync(string to, string
                        subject, string body);
}
```



Benefits: Cleaner controllers



Better testing



Reusable email logic



Easier maintenance

Email Templates

Email templates help keep messages consistent and reusable.

Example 1: Welcome Email

Standard text, user name variable, clean design.



Example 2: Password Reset

A simple link and clear instructions.



Account notification Example 3: Order Confirmation

Order details, shipping status, and thank you message.



 **Key idea:** Use templates instead of building large HTML strings directly in the code.

MailKit Security Best Practices

Securing your MailKit configuration and integration.



**1. Use TLS
or SSL**



**2. Store
credentials
securely**



**3. Do not
hardcode
passwords**



**4. Use app
secrets or
environment
variables**



**5. Validate
recipient
addresses**



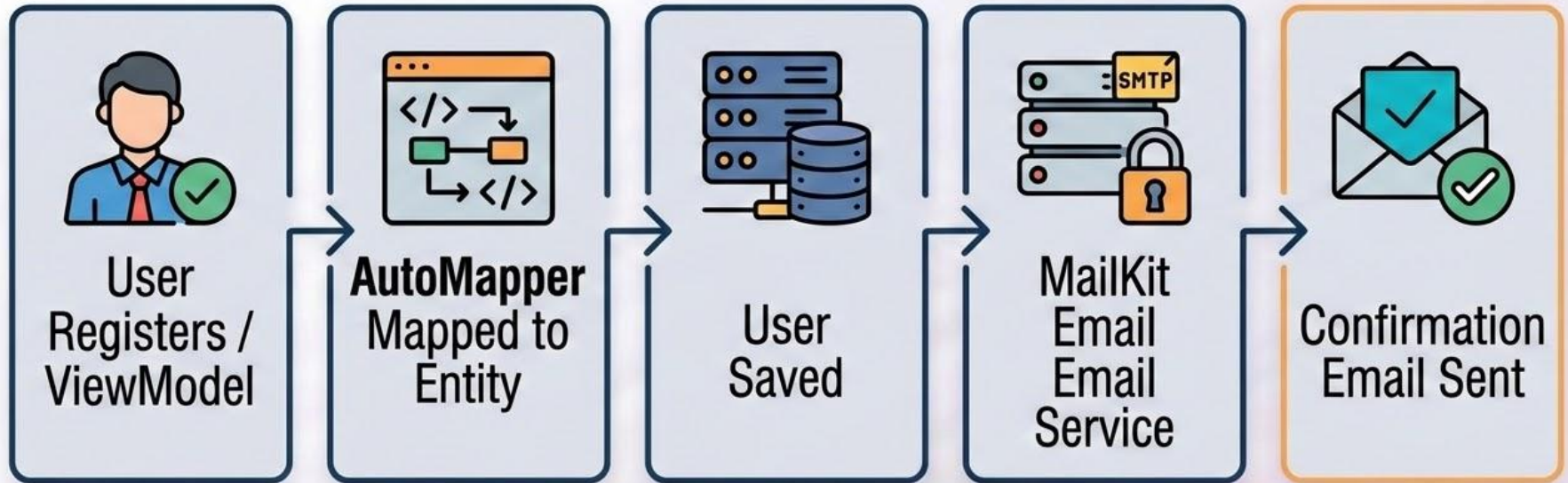
Do Not Share
**6. Avoid
exposing
internal
errors
to users**



Key idea: Implement multi-layered security measures, including encryption, secure storage, input validation, and proper error handling.

Putting It All Together

Example workflow: User registers → ViewModel is mapped to Entity → User is saved → Email service sends confirmation email.



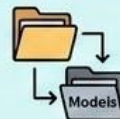
Key Takeaways

Slide 20

AutoMapper simplifies object transformation, while **MailKit** enables reliable email delivery in .NET applications.

Remember:

Use DTOs and ViewModels to separate responsibilities
Distinct models for data transfer & view representation



Remember:

Keep mapping logic organized in profiles



Remember:



Use MailKit through an email service

Remember:



Store email credentials securely

Remember:



Keep controllers **clean** and focused



Implement these practices for efficient and secure applications.